ENGINEERING SOCIETY · TECHNICAL TEAM

UC-01 Deep Dive

Member registration and profile. Every field, every situation, the open decisions we still need to make, and the first job for each of us.

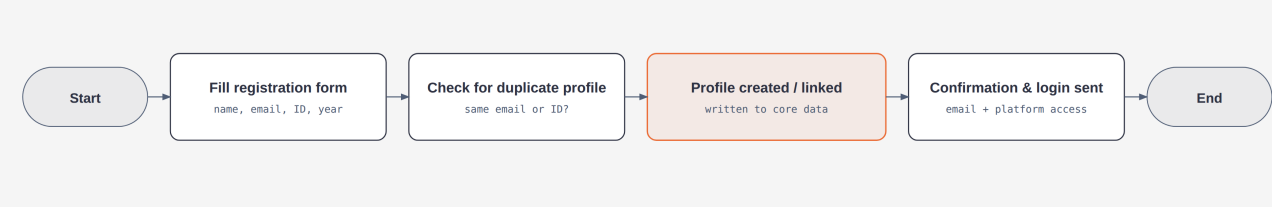
Working document for the first build session

September 2026

PART 1 · THE USE CASE

Member registration, step by step

Actors	Member, Platform
Precondition	Person has a valid email; society registration is open
Trigger	Person opens the registration page
Postcondition	A single, permanent member profile exists and carries forward every future year



1. Fill the registration form

Five fields, matching the Member entity exactly: full name, email, university ID, year, major. Nothing extra is collected here that isn't in the data model, if a future feature needs more, that's a new field added deliberately, not a form growing by accident.

2. Check for a duplicate

Both email and university ID are checked independently. If either one matches an existing profile, this is treated as the same person, not a new one.

3. Create the profile

Written directly to the database. Status is always set to "member" at this point, never "graduate", that transition happens later, in Phase 4, through a separate process.

4. Send the confirmation email

Sent through Resend. The subject and body text are Ibrahim's job, see Part 4.

5. Member logs in

First login uses the credentials set during registration. The profile is now active and permanent.

PART 2 · THE FIELDS

What each field actually means

Some of these have an obvious answer. Some don't, and pretending they do now just means rebuilding the form later. Those are marked as open decisions, not filled in with a guess.

Field	Rule	Status
full_name	Required. No format restriction, must accept apostrophes, hyphens, and non-Latin scripts.	**Settled**
email	Required, unique, valid email format. Compared case-insensitively so Case@x.com and case@x.com are the same person.	**Settled**
	Should this be restricted to a university email domain, or is any email accepted?	**OPEN DECISION**
university_id	Required, unique.	**Settled**
	Exact format (length, numbers only, letters allowed?) isn't defined yet.	**OPEN DECISION**
year	Numeric, the member's current year of study.	**Settled**
	Required at registration, or can it be filled in later?	**OPEN DECISION**
major	Free text.	**Settled**
	Required, or optional?	**OPEN DECISION**
status	System-set to "member" on creation. Not editable by the person registering.	**Settled**

Four open decisions. These get resolved in the first team session, not silently decided by whoever writes the code first.

PART 3 · EVERY SITUATION

What can actually happen

The use case document covers the happy path and two alternate flows. Here's the full picture, organized by what triggers each situation.

Duplicate detection

- Exact email match: link to the existing profile, don't create a new one.
- Same email, different capitalization (Case@x.com vs case@x.com): treated as identical, matched before comparing.
- Same university ID, different email: still the same person, most likely they changed email addresses, link to the existing profile.
- Same name, different email and ID: this is **not** a duplicate. Common names exist. Create a new profile.

Invalid input

- Missing a required field: reject the submission, show exactly which field is missing.
- Malformed email (no @, no domain): reject with an inline error, don't submit to the server at all.
- Empty or whitespace-only name: reject, this isn't a valid name even though it isn't technically "missing."
- Special characters in a name, apostrophes, hyphens, Arabic script: must be **accepted**, not rejected. This is worth stating explicitly so nobody accidentally builds a filter that breaks on real names.

Abandonment and network issues

- Someone closes the tab before submitting: nothing is saved. The form only exists in the browser until submit.
- Submission succeeds but the confirmation page fails to load (dropped connection): the profile was still created. If the person tries again, the duplicate check catches it safely, retrying is never destructive.

After registration

- The confirmation email bounces or lands in spam: the on-screen success message after submitting is the real confirmation, the email is a convenience copy, not the only proof registration worked.
- Someone forgets they already registered and tries again months later: the duplicate check on email or university ID catches this and links to their existing profile instead of creating a second one.

PART 4 · THE FIRST SESSION

One job each, starting now

Salim: test scenario writer

Part 3 lists the situations we already thought of. Your job: go through each of the four categories below and add at least two more real scenarios we haven't listed, then mark whether you think the system as described already handles it, or whether it's a new open decision.

- Duplicate detection, 2 more scenarios
- Invalid input, 2 more scenarios
- Abandonment and network issues, 2 more scenarios
- After registration, 2 more scenarios

Ibrahim: content writer

Every piece of text a member will actually read during registration. One is filled in below as a starting example, the rest are yours to draft.

- Field labels (5): full name, email, university ID, year, major
- **Example, missing-field error:** *"Enter your university email to continue."*
- Missing-field error, for each of the other 4 fields
- Invalid email format error
- Duplicate found message (what does someone see if they already registered?)
- On-screen success message after submitting
- Confirmation email: subject line and full body text

Zaher: user walkthrough

Once a first working version exists, go through it as if you're a real member registering for the first time. Follow the steps below, then report using the template underneath, one entry per issue found.

- Open the registration page on your phone, not a laptop.
- Try registering with a real-looking name, email, and university ID.
- Try submitting with a field left empty.
- Try registering a second time with the same email.
- Check whether the confirmation email actually arrives, and read it as a stranger would.

What I did	
What I expected	
What actually happened	
Severity (blocks launch / annoying / minor)	

PART 5 · DONE MEANS DONE

Definition of done for UC-01

- All four open decisions from Part 2 are resolved and written down here, not left as questions.
- Salim's scenario list has been reviewed and every scenario is either handled or turned into a tracked decision.
- Ibrahim's copy is live in the actual form and email, not placeholder text.
- Zaher has completed the walkthrough with zero unresolved blocking issues.
- The finished build matches the workflow diagram in Part 1 exactly.

Once every box above is checked, UC-01 is done, not before. That's what moves the team to UC-02.